Worksheet Answers: Evaluating LLMs and Code Models

CS 5090 · Software Engineering with Generative AI · Week 8 · answers with short explanations, for your own review. Where a question is open-ended, the answer below is one good answer, not the only one.

Part 1 · Metrics for open-ended tasks, and for code

1.1 *Match examples to the five approaches.*
MMLU = **A** (closed-ended) · BLEU = **B** (reference-based heuristic) · Chatbot Arena = **C** (humans) · AlpacaEval = **D** (LLM as judge) · HumanEval = **E** (execution).

1.2 *Correct vs. higher BLEU.*
Output 1 is **correct**. Output 2 has the **higher BLEU**: it shares almost every token with the reference and differs by three characters, but it drops the first element. BLEU measures overlap with one reference, so a wrong program can score above a correct one that is written differently. For code, the fix is to run it against tests.

1.3 *n = 5, c = 2.*
pass@1 = 1 − C(3,1)/C(5,1) = 1 − 3/5 = **0.40**. pass@2 = 1 − C(3,2)/C(5,2) = 1 − 3/10 = **0.70**. pass^2 = C(2,2)/C(5,2) = 1/10 = **0.10**. A one-answer assistant delivers **pass@1**: pass@k assumes something picks the right sample out of k, which the user does not get.

1.4 *Why two "pass@1 = 70%" may not be comparable.*
Any two of: temperature; n and k; which problems or subset; number of runs or seeds; prompt format and few-shot examples; post-processing such as stripping markdown or stop rules; which test set (HumanEval vs HumanEval+).

Part 2 · Perplexity, leaderboards, Goodhart's law

2.1 *Loss 3.0, $e^3 \approx 20$.*
Perplexity = exp(3.0) ≈ **20**: on average the model is hesitating between about 20 equally likely next tokens. Lower is better; 1 would mean it is always certain.

2.2 *True or false.*
F: perplexity depends on the tokenizer and the test data, so it is only comparable within one setup. **T**: Arena votes are preferences. **T**: the judge is a stronger model acting as the oracle.

2.3 *Length bias, 22.9% → 64.3%.*
(a) Models tuned for the leaderboard became verbose: longer answers won without being better. (b) A length-controlled win rate: model the judge's preference as a function of length and report the win rate as if both answers were equally long; the same verbosity trick then moves the score only from 41.9% to 51.6%, and agreement with Chatbot Arena rises from 0.94 to 0.98. (c) Goodhart's law: once a measure becomes the target, it gets optimized directly and stops measuring what it was meant to measure.

Part 3 · Benchmarks and contamination

3.1 *Benchmark names.*
Real GitHub issues = **SWE-bench Verified** · 164 functions = **HumanEval** · contest problems with release dates = **LiveCodeBench** · same problems, many more tests = **HumanEval+**.

3.2 *88.4 → 76.2.*
(b). The problems and the model are the same; the extra tests catch wrong solutions that the original handful of tests let through. A benchmark score is bounded by the quality of its tests.

3.3 *Contamination detection.*
Before vs after cutoff = **time split** · high probability on the test text = **likelihood** · rewrite the problems = **perturbation** (EvoEval).

3.4 *Three questions for a leaderboard.*
Is there a cutoff date for the test problems, or a private set? Are the test cases hidden? Are the prompt, decoding settings, and agent scaffold reported?

Part 4 · Judges and new capabilities

4.1 *Bias → fix.*
Position → **(c)** run both orders · Verbosity → **(a)** control for length · Self-preference → **(b)** a judge from a different model family.

4.2 *55% agreement on 100 items.*
No. On a two-way judgment 55% is close to chance and far below human–human agreement (about 81%), so the judge is not measuring what you mean. First, write the rubric or reference answer before judging and have the judge apply it. Second, report agreement properly (percent agreement and Cohen's kappa on your hand-labelled items) and use the judge only if agreement is acceptable; otherwise label by hand or change the judge.

4.3 *HumanEvalComm.*
(a) A clarifying question is not code, so it fails every test; pass@k only sees the final program. (b) Communication Rate = **A** (closed-ended: asked or not) · Good Question Rate = **D** (LLM judge) · Pass@1 = **E** (execution).

4.4 *Mean 37.9% vs median 1.6%; EXTREME.*
A few runs had very large gains and most gained little: the distribution is right-skewed, so the mean overstates a typical run. Report both, or the whole distribution. Changes above 9,999% are parameter explosions, not real improvements; counting them as wins would reward broken code, which is Goodhart's law in a continuous objective.

Part 5 · Testing with LLMs

5.1 *Tests that pass on buggy code.*
The model read the implementation and wrote tests that assert what the code does, bug included, so passing proves nothing. The expected values must come from the specification: write the test first, generate tests from the spec rather than the code, or let the model write inputs and setup while you keep the assertions.

5.2 *Mutation score.*
14/20 = **70%**. The assertion-free suite has coverage **100%** and mutation score **0%**, since no mutant can make it fail. Mutation score tells you more: it measures what the tests would catch, coverage only which lines ran.